

低空监管时空数据库设计综合报告

课程名称：空间数据库原理与应用

姓名：云天浩

学号：2024104049

1. 概念设计

1.1 推荐实体清单与空间几何定义

根据需求提取出 11 个核心实体：

实体名称	实体说明	关键属性	空间类型
Drone (无人机)	被监管飞行设备	drone_id, model, owner_org_id, max_height	无
Pilot (飞手)	无人机操控人员	pilot_id, name, license_no, phone	无
Organization (单位)	无人机所属机构	org_id, name, type, contact	无
FlightMission (任务)	申请或执行活动	mission_id, drone_id, pilot_id, status	可含计划范围 Polygon
FlightTrack (轨迹)	任务实际飞行轨迹	track_id, mission_id, avg_altitude, geom	LineString (2D 线)
Telemetry (遥测点)	无人机实时位置	telemetry_id, mission_id, time, altitude, geom	Point (2D 点)
NoFlyZone (管控区)	禁飞/限飞区域	zone_id, name, height_limit, geom	Polygon (2D 面)
TakeoffLandingSite	合法登记起降点	site_id, name, site_type, geom	Point (2D 点)

ViolationEvent (违规)	违规异常事件	event_id, mission_id, type, time, geom	Point (2D 点)
Authority (监管部门)	审批执法行政部门	authority_id, name, level, jurisdiction	无 或 Polygon (2D 面)
DisposalRecord (处置)	执法处置过程记录	record_id, event_id, authority_id, action	无

1.2 空间 E-R 核心关系基数模型

结合 Calkins 空间 E-R 方法定义的实体联系：



2. 逻辑设计

在概念设计的基础上，根据关系数据库设计规范，将 E-R 模型转换为关系模式，并严格遵循**第三范式 (3NF)**，确保消除数据冗余及插入/删除异常。

2.1 关系模式定义

以下为核心实体的关系模式定义，带下划线的属性为**主键 (Primary Key)**，带星号的属性为**外键 (Foreign Key)**：

- Organization (所属单位):** (org_id, name, type, contact)
- Drone (无人机设备):** (drone_id, model, owner_org_id*, max_height, status)
- Pilot (驾驶员/飞手):** (pilot_id, name, license_no, phone)

- **Pilot_Drone (飞手授权映射)**: (pilot_id*, drone_id*) —— 将飞手与飞机的多对多 (M:N) 关系拆解为两个一对多关系。
- **TakeoffLandingSite (起降点)**: (site_id, name, site_type, geom)
- **FlightMission (飞行任务)**: (mission_id, drone_id*, pilot_id*, start_site_id*, end_site_id*, start_time, end_time, status, planned_area)
- **FlightTrack (飞行航线)**: (track_id, mission_id*, avg_altitude, max_altitude, geom)
- **Telemetry (遥测点)**: (telemetry_id, mission_id*, time, altitude, speed, geom)
- **NoFlyZone (时空管制区)**: (zone_id, name, zone_type, height_limit, valid_from, valid_to, geom)
- **Authority (监管执法部门)**: (authority_id, name, level, jurisdiction)
- **ViolationEvent (违规事件)**: (event_id, mission_id*, type, time, severity, geom)
- **DisposalRecord (处置记录)**: (record_id, event_id*, authority_id*, action, result, time)

2.2 数据规范化分析

本设计中，所有非主属性均完全依赖于候选键，且不存在传递依赖。例如，表 Drone 仅通过 owner_org_id 关联企业，而未直接将企业名称和联系人放在飞机表中。这样设计彻底避免了“企业采购飞机前无法录入企业信息”的插入异常与名称的重复存储。

```
-- 1. 先建立独立的企业档案表（彻底消除传递依赖）
CREATE TABLE organization (
    org_id serial PRIMARY KEY,
    name varchar(100) NOT NULL UNIQUE,
    type varchar(50) NOT NULL,
    contact varchar(100) NOT NULL
);

-- 2. 无人机表仅通过 owner_org_id 引用企业主键，不冗余存储企业名称
CREATE TABLE drone (
    drone_id serial PRIMARY KEY,
    model varchar(50) NOT NULL,
    owner_org_id integer NOT NULL, -- 外键关联
    max_height real NOT NULL,
    CONSTRAINT fk_drone_org FOREIGN KEY (owner_org_id)
        REFERENCES organization(org_id) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

3. 物理设计

PostgreSQL 15 + PostGIS 3.

3.1 核心建表代码与完整性约束展示

在物理建表过程中，不仅定义了空间数据类型，更在表级别深度集成了严格的**实体完整性、参照完整性与用户自定义 Check 约束**（例如时态防穿越、高度非负拦截）。以下为关键实体的 SQL 建表核心代码截图/展示：

```
-- 1. 飞行任务表（申请空域与排期校验）
CREATE TABLE flight_mission (
    mission_id serial PRIMARY KEY,
    drone_id integer NOT NULL REFERENCES drone(drone_id) ON DELETE RESTRICT,
    pilot_id integer NOT NULL REFERENCES pilot(pilot_id) ON DELETE RESTRICT,
    start_time timestamp with time zone NOT NULL,
    end_time timestamp with time zone NOT NULL,
    planned_area geometry(Polygon, 4326), -- 计划范围面要素 (WGS 84)
    CONSTRAINT chk_mission_time CHECK (end_time > start_time) -- 时序严谨性校验
);

-- 2. 遥测点表（高频时空轨迹落库点）
CREATE TABLE telemetry (
    telemetry_id bigserial PRIMARY KEY, -- 采用 bigserial 防海量数据溢出
    mission_id integer NOT NULL REFERENCES flight_mission(mission_id) ON DELETE CASCADE,
    time timestamp with time zone NOT NULL,
    altitude real NOT NULL CONSTRAINT chk_alt CHECK (altitude >= 0), -- 严防海拔报错
    geom geometry(Point, 4326) NOT NULL -- 瞬时坐标系地理点 (WGS 84)
);
```

3.2 空间参考系与数据类型选型考量

SRID 4326 (WGS 84)：采用了全球通用的 GPS 经纬度坐标系。选用 `geometry` 类型而非 `geography` 类型，是为了兼容更丰富的高级二维空间算子（如基于外包框的极速相交比对），并通过在算法侧进行投影转换（Cast）来保障测距查询的米级精度。

几何类型映射：无人机瞬时遥测轨迹采用了 `Point` 点集、整体执行航线化简为 `LineString` 线元、禁飞管制空域则抽象为由外部多边形（`Exterior Ring`）构成的 `Polygon` 面元。

3.3 时空查询优化与高并发索引架构

针对“轨迹穿插包含判定”这种极其消耗 CPU 计算资源的痛点，以及管理部门监管面板秒级响应的需求，系统在物理底层设计了 **GiST 空间索引 (R-Tree 变体)** 与 **B-Tree 时间复合索引** 的立体加速架构：

```
-- 纯空间二维拓扑查询优化（提升 ST_Intersects 百倍效率）
CREATE INDEX idx_nofly_geom ON nofly_zone USING gist(geom);
CREATE INDEX idx_telemetry_geom ON telemetry USING gist(geom);

-- 纯时间维度范围查询优化
CREATE INDEX idx_mission_time ON flight_mission(start_time, end_time);

-- 跨列多维联合复合优化（专为“查特定飞机历史轨迹回放”设计，规避全表扫描）
```

```
CREATE INDEX idx_telemetry_mission_time ON telemetry (mission_id, time DESC);
```

4. 系统运行验证

场景一：飞行任务申报冲突检测 (ST_Intersects 时空重叠)

成功检出“任务 2”在飞行申报范围和演唱会临时禁飞区产生了时空交叠冲突。

```
-- =====  
-- 场景一：飞行任务申报冲突检测（面与面时空交叠/相交查询）
```

```
-- 监管逻辑：当飞手或单位申报飞行任务时，系统自动核查申报的计划飞行空域范围（planned_area 面）
--           是否在计划执行时间内（start_time 至 end_time）穿过了设立的禁飞管制空域（nofly_zone
--           面）。
-- PostGIS 算子：ST_Intersects（空间多边形相交） + 边界时间窗口重叠
-- =====
SELECT
    fm.mission_id AS 任务 ID,
    d.model AS 无人机型号,
    p.name AS 执飞飞手,
    fm.purpose AS 任务目的,
    n.name AS 冲突管制区,
    n.zone_type AS 管制类型,
    fm.start_time AS 计划开始时间,
    fm.end_time AS 计划结束时间
FROM flight_mission fm
JOIN drone d ON fm.drone_id = d.drone_id
JOIN pilot p ON fm.pilot_id = p.pilot_id
JOIN nofly_zone n ON ST_Intersects(fm.planned_area, n.geom) -- 空间面与面相交
WHERE n.zone_type = '临时活动管制区'
      -- 时间重叠：任务申报时间与管制区间发生交叠
      AND n.valid_to > fm.start_time
      AND n.valid_from < fm.end_time;
```

输出结果：

场景二：实时遥测轨迹侵入禁飞区警报

成功利用经纬度包含算子抓到了张三。

```
-- =====
-- 场景二：实时遥测轨迹侵入禁飞区警报（动态电子围栏监测）
-- 监管逻辑：无人机上报实时遥测坐标点。系统检索在当前时刻，有哪些无人机的遥测位置（geom 点）
--           落入了正处于生效管制时间内的禁飞区（geom 面）内部，且高度也在其限高范围内。
-- PostGIS 算子：ST_Contains（多边形面包含地理点）
-- =====
SELECT
    t.telemetry_id AS 遥测记录 ID,
    p.name AS 操控飞手,
    d.model AS 无人机型号,
    n.name AS 侵入管制区,
    t.altitude AS 当前上报高度_米,
    n.height_limit AS 管制高度_米,
    t.time AS 侵入时间
```

```

FROM telemetry t
JOIN flight_mission fm ON t.mission_id = fm.mission_id
JOIN pilot p ON fm.pilot_id = p.pilot_id
JOIN drone d ON fm.drone_id = d.drone_id
JOIN nofly_zone n ON ST_Contains(n.geom, t.geom) -- 空间包含关系
WHERE t.time BETWEEN n.valid_from AND n.valid_to -- 时间处于管制内
      AND t.altitude <= n.height_limit; -- 高度在管制层内

```

输出结果：

场景三：遥测高度超限违规检测

不仅抓到了顺丰无人机，还抓到了张三的飞机（其物理限高 120m，实际飞到 150m，也超限了）。

```

-- =====
-- 场景三：遥测高度超限违规检测（超出无人机物理设计高度上限）
-- 监管逻辑：每个无人机设备在注册时都设定了物理设计最大飞行高度（drone.max_height）。
--           当其实时上报的海拔高度（telemetry.altitude）大于设备最大安全升限时，即判定为高度越界违规。
-- 关联关系：Telemetry -> FlightMission -> Drone -> Organization
-- =====
SELECT
    t.telemetry_id AS 遥测点 ID,
    d.model AS 飞行器型号,
    o.name AS 所属单位,
    t.altitude AS 实时上报高度_米,
    d.max_height AS 物理安全高度上限_米,
    (t.altitude - d.max_height) AS 超限差值_米,
    t.time AS 违规时间戳
FROM telemetry t
JOIN flight_mission fm ON t.mission_id = fm.mission_id
JOIN drone d ON fm.drone_id = d.drone_id
JOIN organization o ON d.owner_org_id = o.org_id
WHERE t.altitude > d.max_height; -- 超过物理设计升限

```

输出结果：

场景四：处置与执法大闭环

```
-- =====
-- 场景四：处置与执法闭环综合分析统计（非空间与时空业务大联动）
-- 监管逻辑：对违规异常事件及监管处置结果进行大跨度多表统计分析，
--             呈现单位、飞手、设备、违规详情、执法部门、处罚动作及结果的完整业务闭环。
-- =====

SELECT
    o.name AS 违规单位,
    p.name AS 责任飞手,
    d.model AS 涉事机型,
    ve.type AS 违规事件类型,
    ve.time AS 违规发生时间,
    a.name AS 处置部门,
    dr.action AS 执法措施,
    dr.result AS 处置执行结果,
    dr.time AS 处置办结时间
FROM violation_event ve
LEFT JOIN flight_mission fm ON ve.mission_id = fm.mission_id
LEFT JOIN pilot p ON fm.pilot_id = p.pilot_id
LEFT JOIN drone d ON fm.drone_id = d.drone_id
LEFT JOIN organization o ON d.owner_org_id = o.org_id
JOIN disposal_record dr ON ve.event_id = dr.event_id
JOIN authority a ON dr.authority_id = a.authority_id
ORDER BY ve.time DESC;
```

输出结果：

--